

Índice

1. INTRODUCCIÓN A LA POO	3
1.1. CLASES VS OBJETOS	4
1.2. ABSTRACCIÓN.....	4
1.3. TIPOS DE DATOS ABSTRACTOS	5
1.4. ENCAPSULACIÓN	6
1.5. HERENCIA.....	7
1.5.1. Herencia Simple	7
1.5.2. Herencia Múltiple	8
1.6. POLIMORFISMO	8
2. CLASES.....	10
2.1. CLASES VS ESTRUCTURAS.....	10
2.2. CONSTRUCCIÓN DE UNA CLASE	12
2.2.1. Declaración de una Clase	12
2.2.2. Cuerpo de una Clase	12
2.2.2.1. TIPOS.....	13
2.2.2.2. CONSTANTES.....	13
2.2.2.3. CAMPOS Y PROPIEDADES	14
2.2.2.4. MÉTODOS	16
2.2.2.5. NIVELES DE ACCESIBILIDAD	17
2.2.2.6. MÉTODOS HEREDADOS	19
2.2.2.7. CLASES SELLADAS.....	21
2.2.2.8. CLASES ESTÁTICAS	22
2.2.2.9. MÉTODOS DE EXTENSIÓN.....	24
2.2.2.10. CONSTRUCTORES	25
2.2.2.11. FINALIZADORES (DESTRUCTORES).....	27
2.2.2.12. SOBRECARGA DE OPERADORES.....	28
3. INTERFACES	30
3.1. INTERFACES VS CLASES ABSTRACTAS	32
4. CONCEPTOS AVANZADOS.....	35
4.1. TIPOS DINÁMICOS	35
4.2. ENUMERACIONES	37
4.3. ESPACIOS DE NOMBRES	39
4.4. INSTRUCCIONES USING	39
4.5. TUPLAS	40
4.6. BOXING Y UNBOXING	42
4.7. GENÉRICOS	43
5. TRATAMIENTO DE EXCEPCIONES	47
5.1. TRATANDO EXCEPCIONES	48
5.2. CREANDO NUESTRAS PROPIAS EXCEPCIONES	50

6. ARCHIVOS Y STREAMS.....	51
6.1. TRABAJANDO CON MEMORYSTREAMS	52
6.2. TRABAJANDO CON ARCHIVOS	55

1. INTRODUCCIÓN A LA POO

Los lenguajes de programación siempre se han diseñado en torno a dos conceptos fundamentales. Los datos y el código que opera sobre el dato. Los lenguajes han evolucionado a lo largo del tiempo para cambiar el modo en que estos dos conceptos interactúan.

En un principio, lenguajes como *Pascal* y *C* invitaban a los programadores a escribir software que tratara al código y a los datos como dos cosas separadas, sin ninguna relación. Este enfoque obligaba al programador a traducir al mundo real lo que él quería modelar usando un modelo específico para ordenadores que no era muy amigable.

Estos lenguajes se construyeron en torno al concepto de *procedimiento*. Un procedimiento es un bloque de código con nombre, exactamente igual que los actuales métodos de C#. El estilo de software desarrollado usando estos lenguajes se llama **programación procedural o procedimental**. En la programación procedural, el programador escribe uno o más procedimientos y trabaja con un conjunto de variables independientes definidas en el programa. Todos los procedimientos pueden verse desde cualquier parte del código de la aplicación y todas las variables pueden ser manipuladas desde cualquier parte del código. Sin embargo, en la **POO (Programación Orientada a Objetos)**, por los *niveles de accesibilidad* que podemos darle a cada procedimiento o variable, esto no va a ocurrir.

El desarrollo de software orientado a objetos tiene dos claras **ventajas** sobre el desarrollo de software procedimental:

1. Se puede especificar lo que debe hacer el software y cómo lo hará, usando un *vocabulario familiar* a los usuarios sin preparación técnica. El software se estructura usando **objetos**. Estos objetos pertenecen a las clases con las que el usuario del mundo de los negocios, al que está destinado, está familiarizado. Por ejemplo, durante el diseño de software de un banco puedes modelar clases como *cuenta bancaria, cliente o renta*. Son conceptos que el usuario sabe manejar, pues es conocedor de ese negocio y de la función que cumplen en él. En el estilo procedimental no existen entidades propias que se asemejen a esos modelos como ocurre en la POO.
2. El hecho de que ahora podamos tener ámbitos de nivel de clases, permite ocultar variables en las definiciones de clase. Cada objeto tendrá su propio conjunto de variables y estas variables por lo general solamente serán accesibles mediante las operaciones definidas por la clase. A este principio se le llama *encapsulación*.

El desarrollo de software orientado a objetos se fue haciendo más popular a medida que los programadores adoptaban este nuevo modo de diseñar software y C# es un lenguaje de POO.

1.1. CLASES VS OBJETOS

Una **clase** sólo es una plantilla para la creación de objetos, formada por un conjunto de *propiedades* y *métodos*. Las clases se utilizan para representar entidades o conceptos. Cada clase es un modelo que define un conjunto de variables que representan el estado y un conjunto de métodos para definir los comportamientos del objeto en cuestión.

Los **objetos** son casos concretos de una clase. Los objetos se construyen usando una clase como plantilla. Cuando se crea un objeto éste gestiona su propio estado. El estado de un objeto puede ser diferente del estado de otro de la misma clase.

Por ejemplo, imagina una clase que define a una persona. Una clase `Persona` va a tener *estado* (quizás, una cadena que representa el nombre propio de la clase persona) y *comportamiento*, mediante métodos como `IrATrabajar()`, `Comer()` e `IrADormir()`. Se pueden crear dos objetos de la clase persona, cada uno con un estado diferente.

Cada objeto creado a partir de una clase se llama **instancia** de una clase.

1.2. ABSTRACCIÓN

Es importante darse cuenta de que el objetivo de la programación no es reproducir todos los aspectos posibles del mundo real de un concepto dado. Por ejemplo, cuando se programa una clase `Persona`, no se intenta modelar todo lo que se conoce sobre una persona. En su lugar, se trabaja dentro del contexto de la aplicación específica. Sólo se modelan los elementos que son necesarios para esa aplicación. Algunas características de una persona como la nacionalidad pueden existir en el mundo real, pero se omiten si no son necesarias para la aplicación en cuestión. Este concepto recibe el nombre de **abstracción** y es una técnica necesaria para el manejo de los conceptos infinitamente complejos del mundo real. Por tanto, cuando nos hagamos preguntas sobre objetos y clases, han de ser en el contexto de una aplicación específica.

1.3. TIPOS DE DATOS ABSTRACTOS

Los tipos de datos abstractos fueron el primer intento de determinar el modo en que se usan los datos en programación. Los tipos de datos abstractos se crearon porque en el mundo real los datos no se componen de un conjunto de variables independientes. El mundo real está compuesto de conjuntos de datos relacionados. El estado de una persona para una aplicación determinada puede consistir en, por ejemplo, *nombre, apellidos y edad*. Cuando se quiere crear una persona en un programa, lo que se quiere crear es un conjunto de estas variables. Un tipo de datos abstracto permite presentar tres variables como una unidad y trabajar cómodamente con esta unidad para contener el estado de una persona.

Cuando se asigna un tipo de datos a una variable del código, se puede usar un tipo de datos **primitivo** o un tipo de datos **abstracto**. Los tipos de datos primitivos son tipos que C# reconoce en cuanto se instala. Tipos como `int`, `long`, `char` y `string` son *tipos de datos primitivos* de C#.

Los **tipos de datos abstractos** son tipos que C# no admite cuando se instalan. Antes de poder usar un tipo de dato abstracto hay que declararlo en el código. Los tipos de datos abstractos se definen en el código en lugar de hacerlo en el compilador de C#.

EJEMPLO:

```
class Program
{
    static void Main(string[] args)
    {
        Persona persona = new Persona();
        persona.Nombre = "Jesús";
    }
}
```

Si escribimos código C# que use la clase `Persona` sin indicarle al compilador dónde tenemos declarada la clase, se producirá un error de compilación (a no ser que la tengamos declarada en el mismo espacio de nombres que el programa principal).

Tras definir un tipo de datos abstracto, puede usarse en el código C# exactamente igual que si fuera un tipo de datos primitivo. Las estructuras y las clases de C# son ejemplos de tipos abstractos. Una vez que se ha definido nuestra clase, se pueden usar las variables de ese tipo dentro de otra clase.

1.4. ENCAPSULACIÓN

Mediante la **encapsulación**, los datos se ocultan, o se encapsulan, dentro de una clase y la clase implementa un diseño que permite que otras partes del código accedan a esos datos de forma eficiente. Imaginemos la encapsulación como un envoltorio protector que rodea a los datos de las clases de C#.

EJEMPLO:

```
public class Point
{
    public int X;
    public int Y;
}
```

La clase `Point` representa un punto que se va a pintar dentro de una pantalla, con lo que necesitamos la coordenada `X` y la coordenada `Y`. Ambos datos, es decir, las **propiedades** de esta clase son públicas, lo que permite que cualquier parte del código pueda acceder a ellas y modificar sus valores.

No obstante, puede surgir un problema al permitir que los usuarios asignen los valores a las propiedades directamente. Supongamos que sólo tiene sentido permitir al código que asigne a `X` valores menores de 100 y a `Y` valores mayores de 50. No obstante, con el acceso público a los miembros de datos, no hay nada que impida al código asignar a `X` el valor 32.000 y a `Y` el valor 38.000. El compilador de C# lo permite porque esos valores son posibles en un entero. El problema es que no tiene sentido permitir valores tan elevados.

La encapsulación resuelve este problema. Básicamente, la solución está en marcar los miembros de **datos** como **privados**, para que el código no pueda acceder a los miembros de datos directamente. A continuación podemos escribir dos **métodos públicos** en la clase `Point` como `SetX()` y `SetY()`. Estos métodos pueden asignar los valores a `X` e `Y` y también pueden contener código que genere un error si se les trata de llamar con parámetros cuyos valores son demasiado grandes.

La técnica consistente en marcar como privados los miembros de datos resuelve el problema de que el código establezca sus valores directamente. Si los miembros de datos son privados, sólo la propia clase puede verlos y cualquier otro código que intente acceder a los miembros de datos genera un error de compilación.

El siguiente código muestra cómo se puede implementar el método `SetX()`:

```
public void SetX(int value)
{
    if (value < 100)
        X = value;
    else
```

```
        throw new ArgumentOutOfRangeException();  
    }
```

Este código ha encapsulado el miembro de datos de la coordenada x y permite a los invocadores asignar su valor a la vez que impide que le asignen un valor no válido.

Como los valores de las coordenadas x e y en este diseño son privados, las otras partes de código no pueden examinar sus valores actuales. La accesibilidad privada en la programación orientada a objetos evita que las partes que realizan la llamada lean el valor actual o guarden el nuevo valor. Para exponer estas variables privadas, se pueden implementar métodos como `GetX()` y `GetY()` que devuelven el valor actual de las coordenadas.

1.5. HERENCIA

Algunas clases, como la clase `Point` se diseñan partiendo de cero. El estado y los comportamientos de la clase se definen en ella misma. Sin embargo, otras clases toman prestada su definición de otra clase. En lugar de escribir otra clase de nuevo, se pueden tomar el estado y los comportamientos de otra clase y usarlos como punto de partida para la nueva clase. A la acción de definir una clase usando otra clase como punto de partida se le llama **herencia**.

1.5.1. Herencia Simple

Supongamos que estamos escribiendo código usando la clase `Point` y nos damos cuenta de que necesitamos trabajar con puntos tridimensionales en el mismo código. La clase `Point` que ya hemos definido modela un punto en dos dimensiones y no podemos usarlo para definir un punto tridimensional. Decidimos que hace falta escribir un punto tridimensional llamado `Point3D`. Se puede diseñar esta clase de una de estas dos formas:

1. Se puede escribir la clase `Point3D` partiendo de cero, definiendo miembros de datos llamados x , y y z y escribiendo métodos que lean y escriban los miembros de datos.
2. Se puede derivar de la clase `Point` que ya implementa los miembros x e y . Heredar de la clase `Point` proporciona todo lo necesario para trabajar con los miembros x e y , por lo que todo lo que hay que hacer en la clase `Point3D` es implementar el miembro z .

La primera forma aunque es totalmente válida no es la forma correcta de hacerlo. En su lugar, siempre habría que hacerlo de la segunda forma, es decir, mediante **herencia**.

EJEMPLO:

```
public class Point3D : Point
{
    public int Z { get; private set; }

    public void SetZ(int value)
    {
        if (value < 30)
            Z = value;
        else
            throw new ArgumentOutOfRangeException();
    }

    public int GetZ()
    {
        return Z;
    }
}
```

Como se ve en el ejemplo, a través de los dos puntos estamos diciendo que vamos a heredar de la clase `Point`. Lo que la *clase derivada*, `Point3D`, va a heredar de la *clase base*, `Point`, son todos los métodos y propiedades **públicos** y **protegidos** (visibilidad pública para las clases derivadas y privada para el resto de clases).

A esto se le llama **Herencia simple** porque heredamos únicamente de una clase.

1.5.2. Herencia Múltiple

Algunos lenguajes orientados a objetos también permiten la herencia múltiple, lo que permite que una clase se derive de más de una clase base. C# sólo permite herencias simples, a no ser que heredemos de **interfaces**, en ese caso sí permite la **herencia múltiple**, lo que se verá más adelante.

1.6. POLIMORFISMO

La herencia permite que una clase derivada redefina el comportamiento especificado para una clase base. Supongamos que creamos una clase base `Animal`. Hacemos esta clase base **abstracta** (no tiene implementación, simplemente es la definición de todos los métodos que van a tener que implementarse al heredar de esa clase abstracta) porque queremos codificar animales genéricos siempre que sea posible, pero también crear animales específicos, como un gato y un perro. El

siguiente fragmento de código muestra cómo declarar la clase con su método abstracto.

EJEMPLO:

```
public abstract class Animal
{
    public abstract void HacerRuido();
}
```

Ahora se pueden derivar animales específicos, como Gato y Perro, a partir de la clase base abstracta Animal:

```
public class Gato : Animal
{
    public override void HacerRuido()
    {
        Console.WriteLine("El gato hace miiiauu");
    }
}
public class Perro : Animal
{
    public override void HacerRuido()
    {
        Console.WriteLine("El perro hace Guau Guau");
    }
}
```

Obsérvese que cada clase tiene su propia implementación del método `HacerRuido()`. Ahora la situación es la indicada para el polimorfismo. Tenemos una clase base con un método que está anulado en dos (o más) clases derivadas. El **polimorfismo** es la capacidad que tiene un lenguaje orientado a objetos de llamar correctamente al método anulado en función de que clase lo esté llamando. Esto suele producirse cuando se almacena una colección de objetos derivados.

El siguiente fragmento de código crea una colección de animales, que recibe el apropiado nombre de zoo. A continuación se añade un perro y un gato a este zoo:

```
//Creo una instancia de la clase Gato
Gato gato = new Gato();

//Creo una instancia de la clase Perro
Perro perro = new Perro();

//Creo una lista de Animales y añado al gato y perro
List<Animal> zoo = new List<Animal>();
zoo.Add(gato);
zoo.Add(perro);
```

En este ejemplo de herencia se muestra una clase base y dos clases derivadas. La colección zoo es una colección polimórfica porque todas sus clases derivan de la clase abstracta `Animal`. Ahora se puede iterar la colección y hacer que cada animal emita el sonido correcto:

```
//Recorro la lista de animales y llamo al método HacerRuido()  
//de cada elemento de la lista  
foreach (var animal in zoo)  
{  
    animal.HacerRuido();  
}  
  
Console.ReadKey();
```

Si ejecutamos el código anterior producirá el siguiente resultado:

```
El gato hace miiiauu  
El perro hace Guau Guau
```

¿Qué está pasando? Como C# permite el polimorfismo, en el tiempo de ejecución el programa es lo suficientemente inteligente como para llamar a la versión *perro* de `HacerRuido()` cuando se obtiene un perro del zoo y la versión *gato* de `HacerRuido()` cuando se obtiene un gato del zoo.

En resumen, el **polimorfismo** permite en tiempo de ejecución llamar a la versión de la clase instanciada cuando se obtiene el tipo de clase.

2. CLASES

En el apartado anterior tratamos los conceptos orientados a objetos en general, sin estudiar cómo se usan estos conceptos en el código C#. En el resto de capítulos de este tema explicaremos cómo se implementan estos conceptos en C#.

2.1. CLASES VS ESTRUCTURAS

Las clases y estructuras se parecen mucho. Ambas se pueden definir de manera similar, disponen de métodos y propiedades, se pueden instanciar...

La principal diferencia entre una estructura y una clase en .Net es que las **estructuras** son *tipos por valor* y las **clases** son *tipos por referencia*. Es decir, aunque las estructuras pueden trabajar como clases, realmente son valores ubicados en la pila directamente y no referencias a la información que tienen en memoria.

Esto significa que las estructuras se pueden gestionar más eficientemente al instanciarlas porque se realiza más rápidamente, sobre todo cuando manejamos grandes cantidades de datos, como puede ser una matriz o un listado con miles de elementos. Al crear ese listado de estructuras éstas se crean consecutivamente en memoria y no es necesario instanciar una a una y guardar sus referencias en la matriz, por lo que es mucho más rápido su uso. Además, si pasas a un método una estructura, al hacerlo se crea una copia de la misma como ocurría con los parámetros pasados por valor en los métodos. En consecuencia, los cambios aplicados sobre ella se pierden al terminar de ejecutarse.

EJEMPLO:

```
static void Main(string[] args)
{
    //Creo un instancia de la clase Point
    Point punto = new Point();
    punto.X = 10;
    punto.Y = 8;

    //Creo una instancia de la estructura PointStruct
    PointStruct puntoStruct = new PointStruct();
    puntoStruct.X = 20;
    puntoStruct.Y = 7;

    SumaCoordenadas(punto);
    SumarCoordenadas(puntoStruct);

    Console.WriteLine($"Suma de coordenadas clase: X={punto.X} Y={punto.Y}");
    Console.WriteLine($"Suma de coordenadas struct: X={puntoStruct.X}
        Y={puntoStruct.Y}");
    Console.ReadKey();
}

//Método que recibe la clase Point (por referencia)
public static void SumaCoordenadas(Point point)
{
    point.X = point.X + 10;
    point.Y = point.Y + 10;
}

//Método que recibe la estructura PointStruct (por valor)
public static void SumarCoordenadas(PointStruct pointStruct)
{
    pointStruct.X = pointStruct.X + 10;
    pointStruct.Y = pointStruct.Y + 10;
}
```

Lo habitual no es usar estructuras es usar clases, salvo para realizar optimizaciones o cosas muy concretas. Un ejemplo sería cuando tienes que trabajar con una cantidad muy elevada de objetos en memoria. Volviendo a nuestro ejemplo de la estructura de puntos, supongamos que tuviéramos que hacer masivamente transformaciones sobre esos puntos. Estaríamos hablando de una cantidad tremenda de elementos que tendrían que estar en memoria. En ese

caso sí que nos saldría rentable usar estructuras en lugar de clases, aunque insistimos en que no se suelen utilizar mucho y que lo que más se usa son las clases.

2.2. CONSTRUCCIÓN DE UNA CLASE

Hasta ahora hemos estado trabajando con clases, sin explicar muy bien lo que hacíamos. Pero ya ha llegado el momento de profundizar en cómo se construye una clase. Vamos a empezar desde lo más básico, explicando lo que es una clase, todas las propiedades que tiene y en definitiva, todos los conceptos relacionados con clases.

2.2.1. Declaración de una Clase

Aunque ya hemos visto una definición de clase en anteriores apartados, vamos a revisar el diseño de una declaración de clase:

- ❖ *Modificadores de acceso opcionales*
- ❖ La palabra clave `class`
- ❖ Un *identificador* que da nombre a la clase
- ❖ *Información opcional de la clase base*
- ❖ *El cuerpo de la clase*
- ❖ Un punto y coma opcional

La declaración de clase mínima en C# es parecida a lo siguiente:

```
class MyClass
{
}
```

Las llaves delimitan el cuerpo de la clase. Los métodos y variables de clase se colocan entre las llaves.

2.2.2. Cuerpo de una Clase

El cuerpo de la clase es todo lo que esté dentro de las llaves de apertura y cierre de la clase. Puede contener muchos elementos: *tipos, constantes, campos, propiedades, métodos, eventos, indizadores, constructores, destructores y operadores*.

En los siguientes apartados se profundizará en el estudio de casi todos estos elementos. Otros, en cambio, se dejarán para más adelante. Este es el caso de los *eventos e indizadores*.

2.2.2.1. TIPOS

Las clases pueden usar uno de los tipos integrados en C# (`int`, `long` o `char`, por ejemplo) y pueden definir sus propios tipos. Es por ello por lo que las clases pueden incluir declaraciones de otros elementos, como *estructuras o incluso otras clases*.

2.2.2.2. CONSTANTES

Las **constantes** de clase son variables cuyo valor no cambia. El uso de constantes permite que el código resulte más legible porque pueden incluir identificadores que describan el significado del valor para cualquiera que lea el código. Se puede escribir un código como el siguiente:

EJEMPLO:

```
public class Clase
{
    //Constantes
    const int IVA = 9;
    const int IVA7 = 7, IVA11 = 11, IVA21 = 21;

    private int Importe { get; set; }

    public int CalcularImporte(int importe)
    {
        //IVA = 11; //No se puede modificar
        return importe + (importe * IVA);
    }
}
```

La estructura de la declaración de una constante es la siguiente:

- ❖ La palabra clave **const**
- ❖ Un *tipo de dato* para la constante
- ❖ Un *identificador*, que suele ser en mayúsculas.
- ❖ Un signo *igual*
- ❖ El *valor* de la constante

Se pueden declarar una o varias constantes en la misma línea tal y como se aprecia en el ejemplo.

El uso de constantes presenta dos *ventajas*:

1. Se le da valor a las constantes cuando se las declara y se usa su nombre, no su valor, en el código. *Si por alguna razón es necesario cambiar el valor de la constante, sólo hace falta cambiarlo en un sitio:* donde se declara la constante. Si se escribe en las instrucciones de C# el valor real, un cambio en el valor significaría que habría que revisar todo el código y cambiar ese valor.
2. Las constantes indican específicamente al compilador de C# que, a diferencia de las variables normales, *el valor de la constante no puede cambiar*. El compilador de C# emite un error si algún código intenta cambiar el valor de una constante.

2.2.2.3. CAMPOS Y PROPIEDADES

Un **campo** es una variable de cualquier tipo que se declara directamente en una *clase o struct*. Los campos son *miembros de datos* de la clase o estructura que, en términos de programación orientada a objetos, gestionan el estado de la clase o estructura. Se puede acceder a los campos definidos en una clase o estructura mediante cualquier método definido en la misma.

Una *clase o struct* puede tener *campos de instancia*, *campos estáticos* o ambos. Los **campos de instancia** son específicos de una instancia de una clase o estructura. Por el contrario, los **campos estáticos** (*static*) pertenecen al propio tipo (*clase o estructura*) y se comparten entre todas las instancias de ese tipo. Sólo se puede acceder al campo estático mediante el nombre del tipo.

Una **propiedad** es un miembro que proporciona un mecanismo flexible para leer, escribir o calcular el valor de un *campo privado*. A ese campo privado protegido por la propiedad se le denomina **memoria auxiliar o campo de respaldo**.

Las propiedades se pueden usar como si fueran miembros de datos públicos, pero en realidad son métodos especiales denominados **descriptores de acceso**. Esto permite acceder fácilmente a los datos a la vez que proporciona la seguridad y la flexibilidad de los métodos.

Para devolver el valor de la propiedad se usa un descriptor de acceso de propiedad **get**, mientras que para asignar un nuevo valor se emplea un descriptor de acceso de propiedad **set**.

La palabra clave **value** se usa para definir el valor que va a asignar el descriptor de acceso **set**.

Las propiedades pueden ser **de lectura y escritura** (en ambos casos tienen un descriptor de acceso **get** y **set**), **de solo lectura** (tienen un descriptor de acceso **get**, pero no **set**) o **de solo escritura** (tienen un descriptor de acceso **set**,

pero no `get`). Las propiedades de solo escritura son poco frecuentes y se suelen usar para restringir el acceso a datos confidenciales.

Las propiedades simples que no necesitan ningún código de descriptor de acceso personalizado se pueden implementar como definiciones de cuerpos de expresión o como propiedades implementadas automáticamente.

EJEMPLO:

```
public class Clase
{
    //CAMPO
    //sin valor inicial
    public int Campo;

    //Con get y set damos nivel de accesibilidad a las propiedades
    //desde fuera de la clase
    //Podemos obtener y establecer los datos en la propiedad
    //Descriptores de acceso implementados automáticamente
    public string Nombre { get; set; }

    //Podemos obtener y establecer los datos en la propiedad
    //Descriptores de acceso implementados automáticamente
    public string Apellidos { get; set; }

    //Podemos obtener y NO establecer los datos en la propiedad
    public int Edad { get; private set; }

    //CAMPO DE RESPALDO telefono
    private long telefono;
    //FORMA 1
    //PROPIEDAD Telefono y descriptores de acceso
    //public long Telefono
    //{
    //    get => telefono;
    //    set => telefono= value > 99999999? value : 99999999;
    //}

    //FORMA 2
    //PROPIEDAD Telefono y descriptores de acceso
    public long Telefono
    {
        get { return telefono; }
        set
        {
            if (value > 99999999)
                telefono = value;
            else
                telefono = 99999999;
        }
    }
    //con valor inicial
    public int Estado = 1;
}
```

Se puede establecer el valor de un campo de sólo lectura al declarar el campo o en el constructor de la clase. Su valor no puede establecerse en ningún otro momento. Cualquier intento de cambiar el valor de un campo de sólo lectura es detectado por el compilador de C# y devuelve un error.

```
//De SÓLO LECTURA:  
//Se le puede asignar un valor al declararla  
//O también en el constructor  
public readonly string Identificador = "XX";  
  
//Constructor  
public Clase()  
{  
    Identificador = "ABCD";  
    Edad = 10;  
}  
  
//Método que permite modificar la edad  
//a los mayores de edad  
public void Metodo(int edad)  
{  
    if (edad >= 18)  
    {  
        Edad = edad;  
    }  
  
    //ERROR:Aquí no se puede asignar un valor a  
    //la propiedad readonly  
    //Identificador = "XXX";  
}  
}
```

Los campos de sólo lectura se parecen a las constantes en la medida en que se inicializan cuando se crea un objeto de la clase. La principal diferencia entre una constante y un campo de sólo lectura es que el valor de las **constantes** debe establecerse en tiempo de compilación: en otras palabras, deben recibir su valor *cuando se escribe el código*. Los **campos de sólo lectura** permiten establecer el valor del campo *en tiempo de ejecución*. Como se puede asignar el valor de un campo de sólo lectura en un constructor de clase, se puede determinar su valor en tiempo de ejecución y establecerlo cuando el código se está ejecutando.

2.2.2.4. MÉTODOS

Los **métodos** son bloques de código con nombre en una clase. Los métodos proporcionan los *comportamientos* de las clases. Pueden ejecutarse por cualquier otro fragmento de código en la clase y también pueden ejecutarlos otras clases, dependiendo de su nivel de accesibilidad.

2.2.2.5. NIVELES DE ACCESIBILIDAD

Todas las propiedades y métodos de una clase tienen un nivel de accesibilidad que controla si se pueden usar desde otro código del ensamblado u otros ensamblados. Se pueden usar los siguientes modificadores de acceso:

- ❖ **public**: el acceso no está restringido.
- ❖ **private**: el acceso está limitado al tipo contenedor, es decir, a la clase o estructura en que ha sido declarado el miembro en cuestión.
- ❖ **protected**: el acceso está limitado a la clase contenedora o a los tipos derivados de la clase contenedora.
- ❖ **internal**: el acceso está limitado al ensamblado actual.
- ❖ **protected internal**: el acceso está limitado al ensamblado actual o a los tipos derivados de la clase contenedora, aunque estén en otro ensamblado.
- ❖ **private protected**: el acceso está limitado a la clase contenedora o a los tipos derivados de la clase contenedora que hay en el ensamblado actual.

EJEMPLO:

```
namespace EjNivelesAccesibilidad
{
    public class ClaseBase
    {
        //Se puede acceder desde todos lados
        public string Telefono { get; set; }

        //Sólo se puede acceder desde la propia clase
        private int Edad { get; set; }

        //Sólo se puede acceder desde el mismo ensamblado
        internal string Nombre { get; set; }

        //Se puede acceder desde la propia clase y las clases derivadas
        protected string Apellidos { get; set; }

        //Se puede acceder desde el mismo ensamblado y desde las clases
        //derivadas aunque estén en otro ensamblado
        protected internal string Nacionalidad { get; set; }

        //Se puede acceder desde la misma clase y desde las clases
        //derivadas en el mismo ensamblado
        private protected string Direccion { get; set; }

        //Se puede acceder desde todos lados
        public int Sumar(int a, int b)
        {
            return a + b;
        }
    }

    public class ClaseDerivada:ClaseBase
    {
        public ClaseDerivada()
        {
            //Mediante base. intento acceder a los miembros
            //de la clase base

            //NO PUEDO acceder a:
            //Edad porque es PRIVATE en la clase base
            //base.Edad = 15;
        }
    }
}
```

```

        //SI PUEDO acceder a:
        //Telefono porque es PUBLIC en la clase base
        base.Telefono = "957454545";

        //Nombre porque es INTERNAL en la clase base
        base.Nombre = "Marta";

        //Apellidos porque es PROTECTED en la clase base
        base.Apellidos = "Serrano";

        //Nacionalidad porque es PROTECTED INTERNAL en la clase base
        base.Nacionalidad = "España";

        //Direccion porque es PRIVATE PROTECTED en la clase base
        base.Direccion = "Calle Tortuga";
    }
}
class Program
{
    static void Main(string[] args)
    {
        //No puedo acceder a Edad ni a Apellidos
        //Edad es private en ClaseBase
        //Apellidos es protected en ClaseBase
        //Direccion es private protected en ClaseBase

        ClaseDerivada clasedev = new ClaseDerivada();
        clasedev.Nacionalidad = "Española"; //protected internal
        clasedev.Nombre = "Alejandro"; //internal
        clasedev.Telefono = "1122233"; //public

        Console.WriteLine("La suma es: {0}", clasedev.Sumar(6, 7)); //public
        Console.ReadKey();
    }
}
using EjNivelesAccesibilidad;
namespace PruebaAccesibilidad
{
    //Heredo de ClaseBase que está en otro ensamblado (proyecto)
    class ClassDerivada2 : ClaseBase
    {
        public ClassDerivada2()
        {
            base.Apellidos = "Jiménez"; //protected
            base.Nacionalidad = "Rusa"; //protected internal
            base.Sumar(4, 5); //pública
            base.Telefono = "7656"; //pública
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            ClaseDerivada cd = new ClaseDerivada();
            //Solamente puedo acceder al método y propiedad públicos
            //al estar en otro proyecto y no ser una clase derivada
            cd.Sumar(3, 4);
            cd.Telefono = "4327482";
        }
    }
}

```

Por otro lado, ha de tenerse en cuenta lo siguiente:

- ❖ Sólo se permite un modificador de acceso para un miembro o tipo, excepto cuando se usan las combinaciones *protected internal* o *private protected*.

- ❖ No se permiten modificadores de acceso en *espacios de nombres*. Los espacios de nombres no tienen restricciones de acceso.
- ❖ Si no se especifica ningún modificador de acceso en una declaración de miembro, se usa una *accesibilidad predeterminada*.

Para saber más acerca de las accesibilidades predeterminadas visítese esta página: <https://docs.microsoft.com/es-es/dotnet/csharp/language-reference/keywords/accessibility-levels>

2.2.2.6. MÉTODOS HEREDADOS

C# permite que los métodos de la clase base y de las clases derivadas se relacionen de varios modos. C# permite los siguientes métodos:

- ❖ *Métodos virtuales y de reemplazo*
- ❖ *Métodos abstractos*

MÉTODOS VIRTUALES Y DE REEMPLAZO

En ocasiones puede interesarnos que una clase derivada cambie la implementación de un método de una clase base, pero manteniendo el nombre del método. La operación de reimplementar un método de una clase base en una clase derivada recibe el nombre de *reemplazar* el método de clase base.

Para poder reemplazar un método de una clase base hay que tener en cuenta dos requisitos:

1. El método de la clase base debe ser un **método virtual**, lo que se indica mediante palabra clave **virtual** en la declaración del método.
2. El método de la clase derivada debe ser un **método de reemplazo**, o lo que es lo mismo, un método declarado con la palabra clave **override**.

No se puede reemplazar un método de clase base a menos que sea *virtual*. Sin embargo, sí está permitido reemplazar un método *override*, es decir, reemplazar en una clase derivada un método de la clase padre que, a su vez, también es un método de reemplazo en dicha clase.

MÉTODOS ABSTRACTOS

Un **método abstracto** se declara mediante la palabra clave **abstract** y la definición del método seguida por un punto y coma en lugar de un bloque de método normal, ya que los métodos abstractos no tienen ninguna implementación.

Para poder definir un método como abstracto ha de definirse en una **clase abstracta**, que se declara mediante la palabra clave **abstract** y la definición de la clase.

No se pueden crear instancias de una clase abstracta. Su propósito es proporcionar una definición común de una clase base que múltiples clases derivadas pueden compartir.

Las clases derivadas de la clase abstracta deben implementar todos los *métodos abstractos* mediante *métodos de reemplazo* (**override**). Los métodos abstractos son, por tanto, *métodos virtuales*.

En resumen:

- Para que un método pueda ser reemplazado en la clase derivada ha tenido que ser declarado como *abstracto* o *virtual* en la clase base.
- Si se ha declarado como *abstracto*, hay que *implementarlo obligatoriamente* en la clase derivada. En caso contrario, no.
- Y para que sea abstracto tiene que estar declarado en una *clase abstracta*.
- En una clase abstracta se pueden definir *métodos virtuales*.

EJEMPLO:

```
namespace EjVirtualesAbstractas
{
    public abstract class ClaseAbstracta
    {
        //MÉTODO ABSTRACTO
        public abstract int MetodoAbstractoSumar(int y, int z);

        //MÉTODO VIRTUAL ya implementado
        public virtual int MetodoVirtualRestar(int y, int z)
        {
            return y - z;
        }

        //MÉTODO VIRTUAL ya implementado
        public virtual int MetodoVirtualMultiplicar(int y, int z)
        {
            return y*z;
        }
    }

    public abstract class HijaAbstracta : ClaseAbstracta
```

```

{
    //Implemento el método abstracto
    public override int MetodoAbstractoSumar(int y, int z)
    {
        return y + z;
    }
    //Sobreescribo el método virtual y lo convierto en abstracto
    public abstract override int MetodoVirtualRestar(int y, int z);

    //No toco el método MétodoVirtualMultiplicar,
    //aunque podría sobreescribirlo
}

public class Nieta : HijaAbstracta
{
    //Implemento el metodo virtual de la ClaseAbstracta
    //que en HijaAbstracta se definió como abstracto
    public override int MetodoVirtualRestar(int y, int z)
    {
        return z - y;
    }

    //Este método se definió como abstracto en ClaseAbstracta
    //En HijaAbstracta se implementó con un método de reemplazo (override)
    //Si yo quisiera volver a reemplazarlo, podría
    //public override int MetodoAbstractoSumar(int y, int z)
    //{
    //    return y + z + 1000;
    //}
}

class Program
{
    static void Main(string[] args)
    {
        Nieta na = new Nieta();
        //Se implementa en HijaAbstracta
        Console.WriteLine($"Suma: {na.MetodoAbstractoSumar(1, 2)}");
        //Se llama al método implementado en Nieta,
        //porque en HijaAbstracta se define como abstracto
        Console.WriteLine($"Resta: {na.MetodoVirtualRestar(4, 2)}");
        //Se llama al método implementado en ClaseAbstracta
        Console.WriteLine($"Producto: {na.MetodoVirtualMultiplicar(4, 2)}");

        Console.ReadKey();

        //NO se pueden crear INSTANCIAS de una CLASE ABSTRACTA
        //ClaseAbstracta ca = new ClaseAbstracta();
        //HijaAbstracta ha = new HijaAbstracta();
    }
}

```

2.2.2.7. CLASES SELLADAS

Las clases se pueden declarar como **selladas** si se incluye la palabra clave **sealed** antes de la definición de clase. Una *clase sellada* no se puede utilizar nunca como *clase base*, esta es su característica principal. Por esta razón, tampoco puede ser una *clase abstracta*.

Algunas optimizaciones en tiempo de ejecución pueden hacer que sea un poco más rápido llamar a una clase sellada, que es uno de los motivos por el que se pueden usar.

EJEMPLO:

```
namespace EjClasesSelladas
{
    //No puedo heredar de ClaseSealed
    public class Clase //: ClaseSealed
    {
        public string Nombre { get; set; }
    }
    //La clase sellada sí puede heredar
    public sealed class ClaseSealed:Clase
    {
        public int Edad { get; set; }
    }
}
```

2.2.2.8. CLASES ESTÁTICAS

Con el modificador de acceso **static** convertimos una **clase** en **estática** y esto significa lo siguiente:

- Solo contiene *miembros estáticos*.
- No se pueden crear *instancias* de ella.
- *Está sellada* y, por lo tanto, no puede heredarse.
- Los *constructores* tienen que ser también *estáticos*.

Por lo tanto, crear una clase estática es básicamente lo mismo que crear una clase que contiene solo miembros estáticos y un constructor privado. Un constructor privado impide que se creen instancias de la clase. La ventaja de usar una clase estática es que el compilador garantizará que no se creen instancias de esta clase.

Es más habitual declarar una clase no estática con algunos miembros estáticos que declarar toda una clase como estática.

Para usar los miembros estáticos de cualquier clase, sea estática o no, no hace falta instanciar la clase.

Hay varios casos de uso para **static**, uno de los más comunes es servir para declarar *métodos utilitarios generales* los cuales no dependen de una instancia de la clase que hospeda el método. Cuando hablamos de métodos utilitarios, nos

referimos a funcionalidad común la cual vamos a querer utilizar desde distintos lugares de nuestra aplicación.

Por ejemplo, supongamos que queremos calcular la edad de una persona, ¿Dónde podríamos guardar esta funcionalidad? Una opción es en la clase **Persona**, sin embargo, si deseamos calcular la edad de otras cosas, entonces tendría sentido colocar dicha funcionalidad en una clase aparte. Dado que el cálculo de edad no tiene por qué depender de una instancia de una clase, lo podemos colocar en una clase estática de utilidades, tal y como se ve en el siguiente código:

```
using static System.Console;

namespace EjClasesEstáticas
{
    class Program
    {
        static void Main(string[] args)
        {
            Persona el = new Persona();
            el.Nombre = "Juan";
            el.FechaNacimiento = new DateTime(1982,6,25);
            el.Altura = 183;
            el.Peso = 97;

            //No podemos setear la Edad
            //el.Edad = 50;

            WriteLine($"{el.Nombre} nació el {el.FechaNacimiento.ToLongDateString()}
y tiene {el.Edad} años.");
            WriteLine($"Su IMC es de {Persona.CalcularIMC(el.Altura, el.Peso):N2}");
            ReadKey();
        }
    }

    //Clase Estática
    public static class UtilidadesDeFechas
    {
        public static int CalcularEdad(DateTime fechaNacimiento)
        {
            var edad = DateTime.Today.Year - fechaNacimiento.Year;
            var temp = new DateTime(DateTime.Today.Year,
                                    fechaNacimiento.Month, fechaNacimiento.Day);

            if (temp > DateTime.Today)
                edad--;

            return edad;
        }
    }

    //Esto no podría hacerlo porque nunca se puede heredar
    //de una clase estática
    //public class Persona:UtilidadesDeFechas
```

```

public class Persona
{
    public string Nombre { get; set; }
    public DateTime FechaNacimiento { get; set; }
    public int Altura { get; set; }
    public int Peso { get; set; }

    //Declaramos la Edad como una propiedad que no se puede setear
    public int Edad
    {
        get
        {
            return UtilidadesDeFechas.CalcularEdad(FechaNacimiento);
        }
    }

    //Método estático
    public static double CalcularIMC(int altura, int peso)
    {
        return peso / Math.Pow((double) altura/100, 2);
    }
}

```

2.2.2.9. MÉTODOS DE EXTENSIÓN

Los **métodos de extensión** permiten *agregar* métodos a los tipos existentes sin crear un nuevo tipo derivado, recompilar o modificar de otra manera el tipo original. Los métodos de extensión son *métodos estáticos*, pero se les llama como si fueran métodos de instancia en el tipo extendido. De hecho, no existe ninguna diferencia aparente entre llamar a un método de extensión y llamar a los métodos definidos en un tipo.

Los métodos de extensión se definen como *métodos estáticos*. Su primer parámetro indica en qué *tipo* funciona el método. El parámetro va precedido del modificador **this**.

Los métodos de extensión únicamente se encuentran dentro del ámbito cuando el espacio de nombres se importa explícitamente en el código fuente con una directiva `using`.

En el ejemplo siguiente se muestra un método de extensión definido para la clase `System.String`, que devuelve el número de palabras que hay en una cadena:

```

using ExtensionMethods; //importo el espacio de nombres

namespace EjMetodosExtension
{

```



```

class Program
{
    static void Main(string[] args)
    {
        string s = "Ejemplo de Métodos de Extensión";
        int num = s.WordCount();
        Console.WriteLine($"La cadena tiene {num} palabras.");
        Console.ReadKey();
    }
}

namespace ExtensionMethods
{
    //Clase estática para mis métodos de extensión
    public static class MyExtensions
    {
        //Defino mi MÉTODO DE EXTENSIÓN
        //Para definir el parámetro pongo:
        //THIS + TIPO a extender + nombre variable
        public static int WordCount(this String str)
        {
            return str.Split(new char[] { ' ', '.', '?' },
                             StringSplitOptions.RemoveEmptyEntries).Length;
        }
    }
}

```

2.2.2.10. CONSTRUCTORES

Un **constructor** de una clase o estructura es un *método que se llama igual que la clase o estructura*, que es *público y sin ningún tipo de retorno*. Es decir, un constructor se declara con el nombre de la clase/estructura y el acceso público para que se pueda acceder.

Los constructores permiten al programador establecer valores predeterminados, limitar la creación de instancias y escribir código flexible y fácil de leer.

Puede haber muchos constructores, no tiene que ser un constructor solo. Aunque para que esto sea posible, como mínimo, hemos de definir un constructor sin parámetros. Una vez tengamos este definido, podremos definir otro u más constructores parametrizados.

EJEMPLO:

```

class Program
{
    static void Main(string[] args)
    {

```

```

        //Llamo al constructor por defecto
        Punto punto = new Punto();
        Console.WriteLine($"X = {punto.X} Y = {punto.Y}");

        punto.X = 5;
        Console.WriteLine($"X = {punto.X} Y = {punto.Y}");

        //Al ser value=0, el descriptor set de X le va a asignar x=100
        punto.X = 0;
        Console.WriteLine($"X = {punto.X} Y = {punto.Y}");

        //Llamo al constructor que asigna valores a X e Y
        Punto punto2 = new Punto(10, 20);
        Console.WriteLine($"X = {punto2.X} Y = {punto2.Y}");

        //Llamo al constructor que asigna un valor a X y a Y=0
        Punto punto3 = new Punto(10);
        Console.WriteLine($"X = {punto3.X} Y = {punto3.Y}");

        Console.ReadKey();
    }
}

public class Punto
{
    //Propiedad y descriptores de acceso
    private int x;
    //Bodied member
    public int X {
        //Si value!=0=> x=value, si no x=100
        set => x = value != 0 ? value : 100;
        get => x;
    }

    //-----
    //FORMA 1
    //public int Y { get; set; }

    //FORMA 2
    //Propiedad y descriptores de acceso
    private int y;
    public int Y
    {
        get => y;
        set => y = value;
    }
    //-----

    //Constructor por defecto
    public Punto()
    {
        X = 0; //Al ser value=0, el descriptor set de X le va a asignar x=100
        Y = 0;
    }

    //-----
    //Constructor con 2 parámetros
    //public Punto(int x, int y)

```

```

//{
//    X = x;
//    Y = y;
//}

//Constructor con 2 parámetros
//como método de expresión corporal o bodied member
public Punto(int x, int y) => (X, Y) = (x, y);
//-----

//Constructor con 1 parámetro
//public Punto(int x)
//{
//    X = x;
//    Y = 0;
//}

//Constructor con 1 parámetro
//como método de expresión corporal o bodied member
public Punto(int x) => (X, Y) = (x, 0);
}

```

2.2.2.11. FINALIZADORES (DESTRUCTORES)

Los **finalizadores** (anteriormente conocidos como **destructores**) se usan para realizar cualquier limpieza final necesaria cuando el recolector de basura libera la memoria de los objetos que ya no se utilizan y en consecuencia, los destruye.

Hay que tener en cuenta que:

- ❖ Los finalizadores no se pueden definir en structs. Solo se usan con *clases*.
- ❖ Una clase *solo* puede tener *un finalizador*.
- ❖ Los finalizadores *no* se pueden *heredar ni sobrecargar*.
- ❖ No se puede llamar a los finalizadores. *Se invocan automáticamente*.
- ❖ Un finalizador *no permite modificadores ni tiene parámetros*.

Para definir un destructor únicamente hay que escribir el nombre de la clase antecedido del símbolo de *virgulilla* ~ (Alt+126).

EJEMPLOS:

```

class Car
{
    ~Car() //destructor
    {
        // instrucciones de limpieza...
    }
}

```

```

using System;
public class Destroyer
{

```

```
//Método de expresión corporal
~Destroyer() => Console.WriteLine($"Soy el destructor.");
}
```

En general, C# no requiere mucha administración de memoria por parte del desarrollador. Esto es debido a que el recolector de basura de .NET administra implícitamente la asignación y liberación de memoria para los objetos. En cambio, cuando la aplicación encapsule recursos no administrados, como *ventanas, archivos y conexiones de red*, deberíamos usar finalizadores o destructores para liberar dichos recursos.

2.2.2.12. SOBRECARGA DE OPERADORES

C# define el comportamiento de los operadores cuando se usan en una expresión que contiene tipos de datos integrados en C#. Por ejemplo, el operador + suma dos números, el - resta dos números y así con el resto de operadores.

No obstante, C# permite que se pueda definir el comportamiento de la mayoría de los operadores estándar para que puedan usarse en estructuras y clases propias. Es decir, se pueden escribir métodos especiales para definir el comportamiento de la clase cuando los operadores se usan con instancias de dicha clase. Esto es lo que se llama **sobrecarga de operadores**.

Para sobrecargar un operador hay que definir un método **static** con las siguientes características:

- El *tipo devuelto* deseado. Si se trata de un operador aritmético el tipo devuelto será la *clase o estructura* que contiene el método del operador sobrecargado. Si se trata de los operadores true o false, el tipo devuelto será un *bool*.
- La palabra clave **operator**
- El *operador* que se quiere sobrecargar
- La lista de parámetros que especifique *uno o dos parámetros* de la clase o estructura que contiene el método del operador sobrecargado

EJEMPLO:

```
class Program
{
    static void Main(string[] args)
    {
        //Llamo al constructor que asigna valores a X e Y
    }
}
```

```

Punto punto1 = new Punto(200, 50);
Punto punto2 = new Punto(100, 25);
Console.WriteLine($"PUNTO 1: X = {punto1.X} Y = {punto1.Y}");
Console.WriteLine($"PUNTO 2: X = {punto2.X} Y = {punto2.Y}\n");

punto1++;
Console.WriteLine($"PUNTO 1++:X = {punto1.X} Y = {punto1.Y}\n");

punto1--;
Console.WriteLine($"PUNTO 1--:X = {punto1.X} Y = {punto1.Y}\n");

punto2 = punto1 + punto2;
Console.WriteLine($"PUNTO 2=PUNTO 1 + PUNTO2\nX = {punto2.X} Y = {punto2.Y}\n");

punto2--;
Console.WriteLine($"PUNTO 2--: X = {punto2.X} Y = {punto2.Y}\n");

//Llamo al constructor que inicializa X e Y a 0
Punto puntoBool = new Punto();
EsPuntoVacio(puntoBool);

puntoBool.X = 10;
puntoBool.Y = 20;
EsPuntoVacio(puntoBool);

Console.ReadKey();
}

//Uso de los operadores true y false aplicados a la clase Punto
public static void EsPuntoVacio(Punto miPunto)
{
    if (miPunto)
        Console.WriteLine($"PUNTO BOOL: X = {miPunto.X} Y = {miPunto.Y}");
    else
        Console.WriteLine($"PUNTO BOOL: punto vacío");
}
}

public class Punto
{
    //Propiedad y descriptores de acceso
    public int X { get; set; }

    //Propiedad y descriptores de acceso
    public int Y { get; set; }

    //Constructor por defecto
    public Punto()
    {
        X = 0;
        Y = 0;
    }

    //Constructor con 2 parámetros
    //como método de expresión corporal o bodied member
    public Punto(int x, int y) => (X, Y) = (x, y);

    //Constructor con 1 parámetro

```

```

//como método de expresión corporal o bodied member
public Punto(int x) => (X, Y) = (x, 0);

//Sobrecarga del operador ++
public static Punto operator ++ (Punto p)
{
    Punto nuevoPunto = new Punto();
    nuevoPunto.X = p.X + 1;
    nuevoPunto.Y = p.Y + 1;

    return nuevoPunto;
}

//Sobrecarga del operador --
public static Punto operator -- (Punto p)
{
    Punto nuevoPunto = new Punto();
    nuevoPunto.X = p.X - 1;
    nuevoPunto.Y = p.Y - 1;

    return nuevoPunto;
}

//Sobrecarga del operador +
public static Punto operator +(Punto p1, Punto p2)
{
    Punto nuevoPunto = new Punto();
    nuevoPunto.X = p1.X + p2.X;
    nuevoPunto.Y = p1.Y + p2.Y;

    return nuevoPunto;
}

//Sobrecarga de operador true y false
//Si se define uno, el otro también obligatoriamente
public static bool operator true (Punto p) => (p.X!=0 && p.Y!=0) ? true:false;
public static bool operator false(Punto p) => (p.X==0 || p.Y==0)?false : true;
}

```

3. INTERFACES

Una **interfaz** define un contrato. Cualquier *clase o estructura* que implemente ese contrato debe proporcionar una implementación de los miembros definidos en la interfaz. Mediante las interfaces se puede incluir, por ejemplo, el comportamiento de varios orígenes en una clase, es decir, simular la *herencia múltiple*. Esta capacidad es importante en C# porque el lenguaje no admite la herencia múltiple de clases que ya vimos en apartados anteriores. Además se debe de usar una interfaz si desea simular la *herencia de estructuras* porque no pueden heredar de una clase o una estructura base.

A partir de C# 8.0, una interfaz puede definir una *implementación predeterminada de miembros* (miembros con cuerpo). También puede definir miembros *static* para proporcionar una única implementación de funcionalidad común.

Ni los *campos de instancia* ni las *propiedades automáticas de instancia* se admiten en las interfaces. Esta regla tiene un efecto sutil en las declaraciones de propiedad. En una declaración de interfaz, el código siguiente no declara una propiedad implementada automáticamente como hace en una *clase o estructura*. En su lugar, declara una propiedad que no tiene una implementación predeterminada pero que se debe implementar en cualquier tipo que implemente la interfaz:

EJEMPLO:

```
public interface INombre
{
    public string Nombre {get; set;}
}
```

Por convención, las interfaces empiezan con *I* mayúscula y se declaran con la palabra **interface** antes del nombre que las identifica.

Cualquier *clase o estructura* que implemente la interfaz debe implementar todos sus miembros no implementados.

Al igual que ocurre con las clases abstractas, *no se pueden crear instancias* de una interfaz.

EJEMPLO:

```
namespace EjInterfaz
{
    interface IPunto
    {
        // Firmas de propiedades
        int X { get; set; }

        int Y { get; set; }

        double Distancia { get; }
    }

    class Punto : IPunto
    {
        // Constructor
        public Punto(int x, int y)
        {
            X = x;
            Y = y;
        }

        // Implementaciones de propiedades
        public int X { get; set; }
    }
}
```

```

        public int Y { get; set; }

        public double Distancia =>
            Math.Sqrt(X * X + Y * Y);
    }

    class Program
    {
        static void ImprimirPunto(Punto p)
        {
            Console.WriteLine("x={0}, y={1}", p.X, p.Y);
        }

        static void Main()
        {
            Punto p = new Punto(2, 3);
            Console.Write("Mi Punto: ");
            ImprimirPunto(p);
            Console.ReadKey();
        }
    }
    // Salida: Mi Punto: x=2, y=3
}

```

Cuando una lista de tipos base contiene una clase e interfaces base, la *clase base* debe aparecer *primero en la lista*.

3.1. INTERFACES VS CLASES ABSTRACTAS

Si queremos tener una implementación base de la que hereden otras clases y lo único que queremos es que sobrescriban algunas propiedades y métodos para ajustarlas a sus necesidades, usaremos **clases abstractas**. Dentro de las clases abstractas podemos tener:

- **Miembros implementados** sin poder ser sobrecargados o reemplazados.
- **Miembros abstractos** (sin implementar) y que habrá que sobrecargar (override) en las clases derivadas.
- **Miembros virtuales ya implementados** que podrán ser sobrecargados (override) o no en las clases derivadas.

Si por el contrario, lo que nos interesa es que las clases derivadas tengan que implementar sí o sí ciertos métodos o propiedades usaremos **interfaces**. En este caso, en las clases derivadas tendrán que implementarse los miembros que nunca han sido implementados en la interfaz. Opcionalmente, también podrán implementarse los miembros previamente implementados en la interfaz. En ambos casos, nunca se empleará la palabra `override`.

En el ejemplo creado vamos a ver que se pueden combinar clases abstractas e interfaces.

EJEMPLO:

```
//Clase abstracta
public abstract class Vehiculos
{
    public void Mover()
    {
        Console.WriteLine("Moviendo {0} ruedas del VEHICULO", Ruedas);
    }
    public virtual void Girar()
    {
        Console.WriteLine("Girando VEHICULO");
    }
    public abstract int Ruedas { get; }
}

//Clase derivada de Vehiculos
//Tiene que SOBRECARGAR sólo los miembros abstractos
//de la clase abstracta y opcionalmente los virtuales
//E IMPLEMENTAR todos los miembros no implementados en la interfaz
public class Coche : Vehiculos, IMedTransporte
{
    //Método abstracto de la clase Vehiculos
    public override int Ruedas
    {
        get { return 4; }
    }

    //Propiedades de la interfaz IMedTransporte

    //Esta propiedad está implementada en la interfaz
    //No estoy obligado a implementarla en la clase derivada
    //A no ser que quiera acceder a ella desde una instancia
    //de dicha clase
    //public int Puertas
    //{
    //    get { return 5; }
    //}

    public bool EsAereo
    {
        get { return false; }
    }

    //Método virtual de la clase Vehiculos
    public override void Girar()
    {
        Console.WriteLine("Girando COCHE");
    }
}

//Clase derivada de Vehiculos
//Tiene que SOBRECARGAR sólo los miembros abstractos
//de la clase abstracta y opcionalmente los virtuales
```

```

public class Bicicleta : Vehiculos
{
    public override int Ruedas
    {
        get { return 2; }
    }
}

//Interfaz IVehiculos
public interface IVehiculos
{
    void Mover();
    int Ruedas { get; }
}

//Clase derivada de la interfaz IVehiculos
//Tiene que implementar TODOS los miembros no implementados en la interfaz
public class CocheI : IVehiculos
{
    //Implementa la propiedad Ruedas
    public int Ruedas
    {
        get { return 4; }
    }

    //Implementa el método Mover
    public void Mover()
    {
        Console.WriteLine("Moviendo COCHEI");
    }
}

//Clase derivada de la interfaz IVehiculos
//Tiene que implementar TODOS los miembros no implementados en la interfaz
public class BicicletaI : IVehiculos
{
    //Implementa la propiedad Ruedas
    public int Ruedas
    {
        get { return 2; }
    }

    //Implementa el método Mover
    public void Mover()
    {
        Console.WriteLine("Moviendo BICICLETAI");
    }
}

//Interfaz IMedTransporte
public interface IMedTransporte
{
    int Puertas { get => 2; }
    bool EsAereo { get; }
}

class Program
{
    static void Main(string[] args)
    {
        //ERROR: no se pueden instanciar clases abstractas ni interfaces
    }
}

```

```

//Vehiculos vehiculo = new Vehiculos();
//IVehiculos vehiculoI = new IVehiculos();

Coche micoche = new Coche();
Bicicleta mibici = new Bicicleta();
CocheI micocheI = new CocheI();
BicicletaI mibiciI = new BicicletaI();

Console.WriteLine("----CLASE COCHE--- (derivada de Vehiculo y de la
interfaz IMedTransporte)");
//no se sobrecarga y se llama al método de la clase Vehiculo
micoche.Mover();
//se sobrecarga y se llama al método de la clase Coche
micoche.Girar();

//Esto da ERROR a no ser que implemente Puertas también en la clase
//derivada se llama a la propiedad Puertas implementada en IMedTransporte
//Console.WriteLine($"Es un medio de transporte con
//{micoche.Puertas.ToString()} puertas");

Console.WriteLine("\n----CLASE BICILETA--- (derivada de Vehiculo)");
//no se sobrecarga y se llama al método de la clase Vehiculo
mibici.Mover();
//no se sobrecarga y se llama al método de la clase Vehiculo
mibici.Girar();

Console.WriteLine("\n----CLASE COCHEI--- (derivada de IVehiculo)");
//se implementa y se llama al método de la clase CocheI
micocheI.Mover();

Console.WriteLine("\n----CLASE BICII--- (derivada de IVehiculo)");
//se implementa y se llama al método de la clase BicicletaI
mibiciI.Mover();

Console.ReadKey();
}
}
}

```

4. CONCEPTOS AVANZADOS

4.1. TIPOS DINÁMICOS

Los **tipos dinámicos** se introducen en C# en la versión 4. Se trata de un tipo estático, pero un objeto de tipo `dynamic` omite la comprobación de los tipos estáticos. Es decir, en un tipo estático un entero es un entero, una cadena es una cadena, pero un objeto de tipo dinámico omite esa comprobación. En la mayoría

de los casos funciona como si fuera un tipo `object`. Recordemos que `object` es la base de cualquier clase.

En tiempo de compilación se supone que un elemento de tipo dinámico admite cualquier operación. Por consiguiente, no tendrás que preocuparte de si el objeto obtiene su valor de una API, de un proceso, de una función o de lo que sea, ya que admite todo. Pero si el código no es válido, los errores se detectan en tiempo de ejecución.

EJEMPLO:

```
class Program
{
    static void Main(string[] args)
    {
        ClaseEjemplo ec = new ClaseEjemplo();
        //ERROR. Metodo1 solo admite un parámetro en tiempo de compilación
        //ec.Metodo1(10, 4);

        dynamic dynamic_ec = new ClaseEjemplo();
        //Puedo llamar a Metodo1 como quiera porque es desde un tipo dinámico
        //Sí fallará en tiempo de ejecución
        dynamic_ec.Metodo1(10, 4);

        //Puedo llamar a métodos que tal vez ni existan
        //porque desde los tipos dinámicos no se hace ninguna comprobación
        //en tiempo de compilación
        //para el compilador es como un Object
        dynamic_ec.algunMetodo("algún argumento", 7, null);
        dynamic_ec.metodoNoExistente();

        //A una variable dinámica le puedo asignar lo que yo quiera
        dynamic d = 8;
        //Si realizo una operación con su valor, el resultado también lo
        //considerará como dinámico
        var suma = d + 4;
    }
}

class ClaseEjemplo
{
    //Constructor por defecto
    public ClaseEjemplo() { }
    public ClaseEjemplo(int v) { }

    public void Metodo1(int i) { }

    public void Metodo2(string str) { }
}
```

Las variables dinámicas se utilizan obviamente cuando no sepamos el tipo de dato o de respuesta que vamos a tener en un momento dado para poder tratar

esa variable. Por ejemplo, se puede usar un objeto dinámico para hacer referencia a *Document Object Model (DOM) HTML*, que puede contener cualquier combinación de atributos y elementos de marcado HTML válidos. Si se desea obtener más información al respecto, se puede consultar el siguiente enlace: <https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/types/walkthrough-creating-and-using-dynamic-objects>

4.2. ENUMERACIONES

Un tipo de **enumeración** es un *tipo de valor* definido por un conjunto de *constantes* con nombre del *tipo numérico integral subyacente*. Para definir un tipo de enumeración, usaremos la palabra clave **enum** y especificaremos los nombres de miembros de enumeración:

EJEMPLO:

```
enum Color
{
    Rojo,
    Verde,
    Azul
}
```

Cuando se usa una variable definida como un tipo de enumeración, el compilador de C# se asegura de que los valores asignados a las variables del tipo de la enumeración concuerden con uno de los valores del conjunto de constantes definidas en la definición de la enumeración.

Las enumeraciones son ideales para las situaciones en las que una variable debe asociarse a un conjunto de valores específico.

De forma predeterminada, los valores de constante asociados de miembros de enumeración son del tipo `int`; comienzan con cero y aumentan en uno después del orden del texto de la definición. Aunque se puede especificar explícitamente cualquier otro tipo de número entero como un tipo subyacente de un tipo de enumeración. También se pueden especificar explícitamente los valores de constante asociados, como se muestra en el ejemplo siguiente:

EJEMPLO:

```
//Cambiamos el tipo subyacente a uint
enum Estacion : uint
{
    //Asignamos los valores a cada constante
    Primavera = 1,
    Verano = 2,
    Otoño = 3,
    Invierno = 4
}
```

```
}
```

A continuación vemos el código de nuestro programa de ejemplo:

```
class Program
{
    //Método que recibe una variable del tipo enumerado Color como parámetro
    static void ImprimirColor(Color color)
    {
        switch (color)
        {
            case Color.Rojo:
                Console.WriteLine("Rojo");
                break;
            case Color.Verde:
                Console.WriteLine("Verde");
                break;
            case Color.Azul:
                Console.WriteLine("Azul");
                break;
            default:
                Console.WriteLine("Color desconocido");
                break;
        }
    }
}

static void Main(string[] args)
{
    //Creamos una variable del tipo enumerado Color y le
    //asignamos el valor rojo
    Color c = Color.Rojo;

    //Llamamos al método
    ImprimirColor(c);

    //Llamamos al método pasándole directamente
    //un valor (azul) del tipo enumerado
    ImprimirColor(Color.Azul);

    int i = (int)Color.Azul;    // int i = 2;
    Color c2 = (Color)2;        // Color c = Color.Azul;
    uint est = (uint)Estacion.Invierno; //uint est=4;

    //El método ToString() aplicado a un valor de un enumerado
    //nos devuelve el nombre de dicho valor en formato texto
    string estacion = Estacion.Primavera.ToString();
    Console.WriteLine(estacion);
    Console.ReadKey();
}
}
```

Tal y como puede verse en el ejemplo, para cualquier tipo de enumeración, existen conversiones explícitas entre el tipo de enumeración y su tipo subyacente. Si convertimos (mediante un *casting*) un valor de enumeración a su tipo

subyacente, el resultado es el valor numérico asociado de un miembro de la enumeración.

4.3. ESPACIOS DE NOMBRES

Las clases y estructuras que desarrollemos se pueden encapsular en los llamados **espacios de nombres** (*namespaces*). Los espacios de nombres ayudan a diferenciar unas clases y estructuras de otras que tengan el mismo nombre.

Una clase o estructura completa incluye el nombre del espacio de nombre que la alberga. Cuando se hace referencia a una clase o estructura en un espacio de nombres, hay que calificar el nombre anteponiendo el nombre del espacio de nombres y un punto.

Además, en proyectos grandes, dentro del mismo espacio de nombres podemos tener varias carpetas y dentro de éstas, a su vez, otras clases y estructuras. En este caso, para referenciar a estas clases y estructuras habría que poner la ruta completa. Es decir, el nombre del espacio de nombres seguido de punto, el nombre de la carpeta seguido de otro punto y finalmente el nombre de la clase o estructura.

Para entender mejor el funcionamiento de los namespaces, véase el siguiente vídeo:

https://drive.google.com/file/d/1Yi5zH9C4hNb_XkQ9tpL4L_x5DU0sLWDN/view?usp=sharing

4.4. INSTRUCCIONES USING

La directiva using tiene tres usos:

1. Referenciar a los *namespaces* para incluir clases y poder usarlas.
2. Permitir acceder a los *miembros estáticos* y a los anidados de un tipo. Es decir, poniendo `using static` se puede acceder directamente a los miembros estáticos de la clase referenciada.
3. Crear un **alias** que haga referencia a una clase y así podamos acceder directamente a través de ese alias a dicha clase.

EJEMPLO:

```
//OtroNamespace está dentro de la carpeta Ejemplo
namespace OtroNamespace.Ejemplo
{
```

```

public class ClaseEjemplo
{
    public class Clase1EnEjemplo
    {
        public override string ToString()
        {
            return "Estas en OtroNamespace.Ejemplo";
        }
    }
}

using System; //1er USO
using static System.Math; //2º USO
using static System.Console; //2º USO
using MiAlias = OtroNamespace.Ejemplo.ClaseEjemplo.Clase1EnEjemplo; //3er USO

namespace EjUsing
{
    class Program
    {
        static void Main(string[] args)
        {
            WriteLine($"La raíz cuadrada es {Sqrt(3 * 3 + 4 * 4)}");

            MiAlias instancia = new MiAlias();
            Console.WriteLine(instancia.ToString());
        }
    }
}

```

4.5. TUPLAS

Una **tupla** es una estructura de datos que nos permite almacenar hasta 8 valores diferentes de distintos tipos, que están relacionados de algún modo y usando una sola variable. Es un concepto muy sencillo que posibilita el manejo de manera conjunta de unos pocos datos relacionados sin necesidad de utilizar matrices, colecciones o de crearnos nuestras propias clases para tal manejo.

Las tuplas en C# pueden representarse mediante los tipos de `System.ValueTuple`, o bien, mediante los de `System.Tuple`. La estructura **ValueTuple** es una `Tuple` mejorada, es un tipo de *valor y es mutable*, a diferencia del tipo original. Nosotros siempre trabajaremos con este tipo.

EJEMPLO:

```
using System;
```



```

namespace EjTuplas
{
    class Program
    {
        static void Main(string[] args)
        {
            //Me creo una tupla llamada t
            (int, string, string) t=(1, "Rodrigo", "Sainz Rey");

            //Muestro el valor de cada elemento de la tupla
            Console.WriteLine($"La persona {t.Item1} es {t.Item2} {t.Item3}");

            Console.WriteLine("\nIntroduce una frase:");
            string cadena = Console.ReadLine();

            //Recojo los valores devueltos por el método
            (int numPalabras, string ultPalabra) = SeparadorDeFrases(cadena);
            //Muestro cada uno de esos valores
            Console.WriteLine($"La cadena \"{cadena}\" +
            $"tiene {numPalabras} palabras y la última es {ultPalabra}");

            //Llamo al método que hace lo mismo pero pasándole la tupla
            PrintMsje(cadena, (numPalabras, ultPalabra));

            //Llamo al método que hace lo mismo pero pasándole la tupla
            PrintMsje2(cadena, (numPalabras, ultPalabra));

            Console.WriteLine("\nIntroduce otra frase:");
            string cadena2 = Console.ReadLine();

            //Muestro cada valor devuelto por el método
            Console.WriteLine($"La cadena \"{cadena2}\" +
            $"tiene {SeparadorDeFrases(cadena2).Item1} palabras y la última" +
            $" es {SeparadorDeFrases(cadena2).Item2}");

            //Llamo a un método que recibe un único parámetro
            //de tipo Object y le paso una tupla
            MetodoConUnParametroObject(t);
        }

        //Recibe una cadena como parámetro, la separa en palabras y devuelve
        //El nº de palabras de la cadena y la última palabra
        public static (int, string) SeparadorDeFrases(string cadena)
        {
            string[] palabras = cadena.Split();

            return (palabras.Length, palabras[palabras.Length-1]);
        }

        //Implemento 2 métodos que reciben una cadena y una tupla llamada texto
        //Muestro cada elemento de la tupla por consola

        //FORMA 1, accedo a cada elemento de la tupla por su nombre
        public static void PrintMsje(string cad, (int num, string ultima) tex)
        {
            Console.WriteLine($"---MÉTODO PrintMsje\nLa cadena \"{cad}\" +
            $"tiene {tex.num} palabras y la última es {tex.ultima}");
        }
    }
}

```

```

    }

    //FORMA 2, accedo a cada elemento de la tupla por Item1, Item2...
    public static void ImprimirMensaje2(string cad, (int, string) tex)
    {
        Console.WriteLine($"\\n---MÉTODO PrintMsje2\\nLa cadena \"{cad}\"\\n:\\n" +
            $"tiene {tex.Item1} palabras y la última es {tex.Item2}");
    }

    //Método que recibe un parámetro de tipo object
    public static void MetodoConUnParametroObject (object parametro)
    {
        Console.WriteLine(parametro.ToString());
    }
}

```

Los principales usos de una tupla son:

1. *Manejar en memoria conjuntamente un grupo pequeño de datos que están relacionados.*
2. *Devolver varios valores en un método sin tener que usar una clase propia. Este es el uso más extendido.*
3. *Pasar fácilmente varios parámetros a algún método que sólo admite un parámetro Object.*

4.6. BOXING Y UNBOXING

La conversión **boxing** es el proceso de convertir de forma implícita un *tipo de valor*, sea el que sea, en el tipo `object`.

El proceso de **unboxing** es todo lo contrario, es decir, coger un `object` y obtener de forma explícita el *tipo de valor* de ese `object`.

EJEMPLO:

```

static void Main(string[] args)
{
    int i = 123;

    // BOXING: se copia el valor de i en el objeto o
    object o = i;

    o = 456;
    i = (int)o; // UNBOXING

    int i2 = 123;
    object o2 = i2; // BOXING implícito

    try

```

```

{
    int j = (short)o2; // intento de unbox
    //int j = (int)o2;

    System.Console.WriteLine("Unboxing OK.");
}
catch (System.InvalidCastException e)
{
    System.Console.WriteLine("{0} Unboxing incorrecto.", e.Message);
}
}

```

4.7. GENÉRICOS

Desde la versión 2.0 de C# tenemos soporte para los **genéricos**. Con los genéricos se pueden crear clases o métodos que son independientes de su tipo contenido. En vez de escribir métodos o clases con la misma funcionalidad para diferentes tipos, se puede crear únicamente un método o una clase para ello.

Otra opción para reducir nuestro código sería el uso de la clase **object**. Sin embargo el problema de emplear dicha clase es que no es *type-safe*, es decir no se hace comprobación de tipos. Las clases genéricas hacen uso de tipos genéricos que son reemplazados por tipos específicos cuando se requiera. Esto permite la comprobación de tipos durante la compilación.

Los **genéricos** no sólo se pueden usar en clases sino también se pueden emplear en métodos, interfaces, etc.

Algunas *ventajas* del empleo de genéricos son:

- ❖ **Rendimiento.** Cuando se realizan operaciones de *boxing* y *unboxing* en colecciones con muchos elementos puede producirse una pérdida de rendimiento. Con el empleo de **genéricos**, al pasarle el tipo como parámetro, no hay que realizar el *boxing* y el *unboxing*, y por lo tanto, no hay pérdida de rendimiento.
- ❖ **Seguridad de tipos.** Con los genéricos se pueden detectar los posibles errores en tiempo de compilación, a diferencia de lo que pasa con los *boxing* y *unboxing*.
- ❖ **Reutilización de código.** Una clase genérica se define una única vez y se puede reutilizar con cualquier tipo. Un ejemplo básico es el de la clase `List` del espacio de nombres `System.Collections.Generic` que se puede instanciar como una lista de enteros, de cadenas, de clases...

EJEMPLO:

```
public class Persona
{
    public string Nombre { get; set; }
    public string Apellidos { get; set; }
}

//Clase que se conecta a una API. Si todo va bien recojo una serie de mensajes
//devueltos al conectarme y el objeto persona devuelto
public class OperationResult
{
    //Devuelvo la lista si no está vacía
    public bool Success => !MessageList.Any();

    //Propiedades
    public List<string> MessageList { get; private set; }
    public Persona Persona { get; set; }

    //Constructor
    public OperationResult()
    {
        MessageList = new List<string>();
    }

    //Añado un mensaje a la lista de mensajes
    public void AddMessage(string message)
    {
        MessageList.Add(message);
    }

    //Le asigno una persona a mi propiedad Persona
    public void SetSuccessResponse(Persona pers)
    {
        Persona = pers;
    }
}

public class Car
{
    public string Matricula { get; set; }
}

//Clase que se conecta a una API. Si todo va bien recojo una serie de mensajes
//devueltos al conectarme y el objeto coche devuelto
public class OperationResultCars
{
    public bool Success => !MessageList.Any(); //Devuelvo la lista si no está vacía

    //Propiedades
    public List<string> MessageList { get; private set; }
    public Car Coche { get; set; }

    //Constructor
    public OperationResultCars()
    {
        MessageList = new List<string>();
    }

    //Añado un mensaje a la lista de mensajes
}
```

```

public void AddMessage(string message)
{
    MessageList.Add(message);
}

//Le asigno un coche a mi propiedad Coche
public void SetSuccesResponse(Car coche)
{
    Coche = coche;
}
}

```

Como podemos observar las clases `OperationResult` y `OperationResultCars` son prácticamente iguales, por lo que estamos repitiendo mucho código.

Para convertir nuestro código debemos reemplazar el código que es diferente (*Persona* y *Car* en nuestro caso) por su parte genérica:

```

//Clase genérica
public class OperationResult<T>
{
    public bool Success => !MessageList.Any(); //Devuelvo la lista si no está vacía

    //Propiedades
    public List<string> MessageList { get; private set; }
    public T Response { get; set; }

    //Constructor
    public OperationResult()
    {
        MessageList = new List<string>();
    }

    //Añado un mensaje a la lista de mensajes
    public void AddMessage(string message)
    {
        MessageList.Add(message);
    }

    //Le asigno un objeto del tipo T a mi propiedad Response del tipo T
    public void SetSuccesResponse(T obj)
    {
        Response = obj;
    }
}

```

Por convención, los parámetros de tipo genérico vienen prefijados con la letra `T` y deben ser únicos en la declaración para evitar conflictos de nombres en la implementación.

El tipo `T` aceptará cualquier tipo, tanto uno creado por nosotros como puede ser la clase o tipo *Persona* o un tipo como puede ser el tipo `string`.

Por supuesto podemos implementar más de un tipo en nuestra `generic class`, para ello debemos pasar más de un tipo junto al nombre de la clase. Esto suele usarse cuando se trabaja con *tuplas*. Para nombrar a cada parámetro que va a recibir la clase genérica, por convención, utilizaremos un nombre descriptivo y que siempre empezará por `T`, para indicar a quien lea nuestro código que es un *Tipo*:

EJEMPLO:

```
public class OperationResult<TEntrada, TSalida>
{
    //Code
}
```

Pero las clases no son las únicas partes que pueden ser genéricas en nuestro código, también podemos hacer métodos genéricos. Para ello tenemos dos opciones:

A - Método genérico en clase genérica

Primero podemos tener un método que recibe un parámetro `T` o devuelve un parámetro `T` lo cual es completamente válido, como se ha visto en el ejemplo anterior:

```
public void SetSuccessResponse(T obj)
{
    Response = obj;
}
```

B - Método genérico en clase NO genérica

Si disponemos de una clase no genérica, podemos crear un método que acepte tipos genéricos, quizá no son tan comunes, pero se suelen ver de vez en cuando.

Para tener un método genérico en una clase no genérica lo hacemos de la siguiente manera:

```
public class Ejemplo
{
    public T GenericMethod<T>(T receivedGeneric)
    {
        return receivedGeneric;
    }
}
```

Indicamos después del nombre del método el tipo `<T>` y como vemos podemos devolver también el propio tipo.

En resumen:

- Se deben usar los tipos genéricos para maximizar *la reutilización del código, la seguridad de tipos y el rendimiento*.
- El uso más común de genéricos es crear *clases de colección*.
- La biblioteca de clases .NET contiene varias clases de colección genéricas en el espacio de nombres `System.Collections.Generic`. Estos se deben usar siempre que sea posible en lugar de clases como `ArrayList` en el `System.Collections` espacio de nombres.
- Podemos crear nuestras propias *interfaces genéricas, clases, métodos, ...*

5. TRATAMIENTO DE EXCEPCIONES

Buscar errores y manejarlos adecuadamente es un principio fundamental para diseñar el software correctamente. En teoría escribimos el código y cada línea funciona como pretendemos y los recursos que empleamos siempre están presentes. Sin embargo, este no siempre es el caso en un mundo real: otros programadores pueden cometer errores, las conexiones de red pueden interrumpirse, los servidores de bases de datos pueden dejar de funcionar, los archivos de disco pueden no tener contenidos que las aplicaciones creen que contienen... En pocas palabras, *el código que se escribe tiene que ser capaz de detectar errores como éstos y responder adecuadamente*.

Los mecanismos para informar de errores son tan diversos como los propios errores. Algunos métodos pueden estar diseñados para devolver un valor booleano que indica el éxito o el fracaso de un método, otros métodos pueden escribir errores en un fichero de registro o una base de datos de algún tipo, etc.

La variedad de modelos de presentación de errores nos indica que el código que escribimos para controlar los errores debe ser bastante consistente. Cada método puede informar de un error de un modo distinto, lo que significa que la aplicación está repleta de gran cantidad de código necesario para detectar los diferentes tipos de errores de las diferentes llamadas al método.

.NET Framework proporciona un mecanismo estándar llamado **control de excepciones** para informar de errores. Este mecanismo depende de las

excepciones para indicar los fallos y eso es lo que vamos a ver en los siguientes apartados.

5.1. TRATANDO EXCEPCIONES

Un **control de excepciones** se produce obviamente cuando se produce un *error*. Por ejemplo, imaginemos que queremos acceder a un fichero o un acceso en una ruta que no existe, hacer un casteo de un objeto y no funciona, etc.

Para declarar una excepción tenemos que usar la siguiente sintaxis:

```
try
{
    //instrucciones try
}
catch
{
    //instrucciones catch
}
```

Con la palabra clave **try** estamos especificando que deseamos ser informados sobre la excepción que pueda producirse.

Con la palabra clave **catch** indicamos que queremos atrapar la excepción iniciada y ocuparnos de ella. Por lo tanto, las instrucciones de este bloque sólo se ejecutan si se inicia una excepción desde el bloque `try`. Si ninguna de las instrucciones del bloque `try` inicia una excepción, entonces no se ejecuta nada del código del bloque `catch`.

EJEMPLO:

```
try
{
    //bloque de código
    object obj = "aa";
    int variable = (int)obj;
}
catch (Exception ex)
{
    //capturamos la exception
    //Mostramos el tipo de excepción
    Console.WriteLine(ex.GetType());
    //Motramos el error que se ha producido
    Console.WriteLine(ex.Message);
}
```


Además, el código del bloque `try` puede iniciar diferentes *clases de excepciones*. Si se quiere controlar qué clase de excepción es la que ha sido iniciada, se pueden especificar varios bloques `catch`. Cada uno de ellos controla una clase de error específica:

EJEMPLO:

```
try
{
    //bloque de código

    object obj = "aa";
    int variable = (int)obj;

    int valor = 0;
    int division = 6 / valor;

    int[] matriz = { 0, 1 };
    matriz[2] = 5;

    Console.Write("TEXTO: ");
    int num = Convert.ToInt32(Console.ReadLine());
}
catch (IndexOutOfRangeException ex)
{
    //capturamos la exception
    Console.WriteLine($"catch1 : {ex.GetType()}");
}
catch (DivideByZeroException ex)
{
    //capturamos la exception
    Console.WriteLine($"catch2 : {ex.GetType()}");
}
catch (InvalidCastException ex)
{
    //capturamos la exception
    Console.WriteLine($"catch3 : {ex.GetType()}");
}
catch (Exception ex)
{
    //capturamos la exception
    Console.WriteLine($"catch4 : {ex.GetType()}");
}
finally
{
    //siempre se ejecuta
    Console.WriteLine("Fin del programa");
}
```

Finalmente, los bloques `catch` pueden ir seguidos por otro bloque de código. Este bloque de código se ejecuta después del `catch`, tanto si se produce una excepción como si no. Este bloque se especifica mediante la palabra clave **finally**, tal y como se aprecia en el ejemplo anterior.

5.2. CREANDO NUESTRAS PROPIAS EXCEPCIONES

.NET Framework declara una clase llamada `System.Exception`, que sirve como clase base para todas las excepciones. Las clases predefinidas del entorno común de ejecución derivan de **`System.SystemException`** que, a su vez, deriva de `System.Exception`.

Hay tres excepciones a esta regla que son el `DividedByZeroExceptions`, el `NotFiniteNumberException` y `OverflowException`, que derivan de una clase llamada `System.ArithmeticException`, que también deriva de **`System.SystemException`**.

Para crear nuestra propia excepción lo único que tenemos que hacer es crear una clase que derive de **`System.ApplicationException`** (ésta también deriva de `System.Exception`) con un constructor que llame al de la clase base pasándole un string como parámetro, tal y como se muestra en el siguiente ejemplo:

EJEMPLO:

```
public class MyException : ApplicationException
{
    public MyException(): base("Esta es mi CustomException")
    {
    }
}

class Program
{
    static void Main(string[] args)
    {
        try
        {
            //Llamo a un método que devuelve un 1 si todo OK
            //si no, el método lanza una excepción del tipo
            //MyException
            int variable = Ejemplo();
        }
        catch (MyException ex)
        {
            //capturamos la exception
            Console.WriteLine(ex.Message);
        }
        finally
        {
            //siempre se ejecuta
            Console.WriteLine("Hasta luego");
        }
    }
}
```

```

public static int Ejemplo()
{
    try
    {
        //bloque de código
        object obj = "aa";
        int variable = (int)obj;
    }
    catch (Exception ex)
    {
        //Lanzo mi propia excepción
        //y finaliza el método
        throw new MyException();
    }

    //Si no se ha producido ninguna excepción
    //devuelvo un 1
    return 1;
}
}

```

En resumen, .NET Framework usa las excepciones para informar de diferentes errores a las aplicaciones. El lenguaje C# admite perfectamente el tratamiento de las excepciones y permite al usuario definir sus propias excepciones, además de trabajar con las excepciones definidas por .NET Framework.

La ventaja de usar excepciones reside en que no es necesario comprobar cada llamada de método en busca de un error. Se puede encerrar un grupo de llamadas de método en un bloque `try` y se puede escribir código como si cada llamada de método del bloque tuviera éxito. Esto hace que el código del bloque `try` sea mucho más limpio porque no necesita ninguna comprobación de errores entre líneas. Cualquier excepción iniciada desde el código del bloque `try` es gestionada en un bloque `catch`.

6. ARCHIVOS Y STREAMS

Cuando queremos copiar o trabajar con objetos en memoria, ya sea para pasarlos u obtenerlos de un fichero, o bien, para moverlos de la memoria a un fichero o de un fichero a memoria, tendremos que trabajar con *streams* y copiar el contenido byte a byte.

Un **stream** es un conjunto de bytes, en el cual se puede leer información, escribir datos, situarse en una posición determinada dentro del bloque...

Para trabajar con streams en la memoria nos apoyaremos en una clase conocida como **MemoryStream**. Cuando necesitemos usar esta clase tendremos que agregar el namespace al que pertenece. En este caso, `System.IO`. Esta clase crea un stream y lo guarda como un array de bits sin signo en memoria.

Para crear un *MemoryStream* hay que crear una instancia de dicha clase proporcionándole un tamaño (*capacidad*). Los objetos de tipo *MemoryStream* tienen tres *propiedades* destacables:

- ❖ **Capacity**: nos indica cuántos bytes puede almacenar el *stream* y es de tipo entero. Está limitado al tamaño máximo de un array de bytes, es decir, al número de elementos que puede tener. Si al crear la instancia le pasamos el tamaño, éste ya no se podrá modificar.
- ❖ **Length**: nos indica el tamaño de la información actual que tiene el *stream*, es decir, cuántos bytes estamos usando realmente. Es de tipo `long`.
- ❖ **Position**: este dato es sumamente importante ya que nos indica el lugar donde se encuentra el byte actual, es decir, el siguiente a ser procesado. Esta posición está referenciada con relación al inicio del *stream*. Es de tipo `long`.

EJEMPLO:

```
using System;
using System.IO; //namespace para MemoryStream

namespace EjMemoryStream1
{
    class Program
    {
        static void Main(string[] args)
        {
            //Instancio un MemoryStream con capacity=150
            MemoryStream ms = new MemoryStream(150);

            int capacidad = ms.Capacity; //tamaño máximo que puede tener
            long longitud = ms.Length; //tamaño usado
            long posicion = ms.Position; //posición en la que estoy
        }
    }
}
```

6.1. TRABAJANDO CON MEMORYSTREAMS

Para poder operar con *streams* necesitamos conocer el funcionamiento de los siguientes métodos:

- ❖ **Seek (distancia, puntoReferencia)**: nos permite colocar la posición actual dentro del *stream* a la distancia indicada a partir del

puntoReferencia. Este parámetro tiene que ser de tipo `Seek.Origin`, que es un enumerado con tres valores posibles: `Begin` (estamos referenciando con relación al *origen del stream*), `Current` (estamos referenciando en relación a la *posición actual del stream*) y `End` (estamos referenciando con relación a la *parte final del stream*).

Si no utilizamos correctamente el método **Seek** tendremos problemas con nuestro programa. Un problema común es tratar de colocar la posición actual antes del punto de inicio o después del final del *stream*. Debemos tener una lógica para evitar estas acciones y para lograrlo tenemos que hacer uso de las propiedades del *stream* (`position`, `length` y `capacity`).

❖ **Read(array, desplazamiento, numBytes)**: lee `numBytes` del *stream* a partir de la posición actual (dependerá de donde nos hayamos posicionado con `Seek`) y los guarda en el array. Si `desplazamiento=0`, se guardará en la posición 0 del array. En caso contrario, se guardará en la posición indicada en `desplazamiento`.

La información se lee byte a byte. Cuando hacemos una lectura, el byte que se lee es el que se encuentra en la posición actual (`Origin.Current`). Inmediatamente después de leer, la posición actual se actualiza y nuestra nueva posición actual será la del byte consecutivo al que acabamos de leer.

❖ **Write(array, desplazamiento, numBytes)**: de toda la información que hay en el array se cogen los bytes especificados por `numBytes` y se escriben en el *stream* a partir de la posición dentro del array especificada por el `desplazamiento`.

❖ **Close()**: cierra el stream. Este método ha de utilizarse cuando vayamos a terminar de usar el objeto *MemoryStream* instanciado.

EJEMPLO:

```
using System;
using System.IO; //para MemoryStream
using System.Text; //para ASCIIEncoding

namespace EjMemoryStreamCompleto
{
    class Program
    {
        static void Main(string[] args)
        {
            MemoryStream memStream = new MemoryStream(50);
            string miInformacion = "";

            //Obtengo las propiedades de memStream
            int capacidad2 = memStream.Capacity;
```

```

        long longitud2 = memStream.Length;
        long posicion2 = memStream.Position;

        //Me declaro un array para guardar el texto que voy
        //a leer de memStream
        byte[] miArrayToWrite = new byte[50];
        byte[] miArrayToRead = new byte[50];

        //Pido al usuario que introduzca el texto
        Console.WriteLine("Introduce la información:");
        miInformacion = Console.ReadLine();

        //Guardo el texto en mi array de bytes
        miArrayToWrite = ASCIIEncoding.ASCII.GetBytes(miInformacion);

        //Escribo en el MemoryStream el contenido del array
        //empezando por el byte 0 y terminando en el último byte
        //usado del array
        memStream.Write(miArrayToWrite, 0, miInformacion.Length);

        //HAGO 3 LECTURAS

        //LECTURA1
        //Me posiciono en el principio del MemoryStream
        memStream.Seek(0, SeekOrigin.Begin);
        //Leo los 5 primeros bytes y los guardo en el array
        memStream.Read(miArrayToRead, 0, 5);
        //Muestro el texto que hay en el array
        Console.WriteLine(ASCIIEncoding.ASCII.GetString(miArrayToRead));

        //LECTURA2
        //Avanzo 2 bytes desde la posición actual
        memStream.Seek(2, SeekOrigin.Current);
        //Leo los 5 primeros bytes y los guardo dentro del array
        //en la posición 0
        memStream.Read(miArrayToRead, 0, 5);
        Console.WriteLine(ASCIIEncoding.ASCII.GetString(miArrayToRead));

        //LECTURA3
        //Retrocedo 10 bytes desde la posición final
        memStream.Seek(-10, SeekOrigin.End);
        //Leo 10 bytes y los guardo dentro del array
        //en la posición 0
        memStream.Read(miArrayToRead, 0, 10);
        Console.WriteLine(ASCIIEncoding.ASCII.GetString(miArrayToRead));

        //Cierro el MemoryStream
        memStream.Close();
    }
}

```

6.2. TRABAJANDO CON ARCHIVOS

Los **streams** no solamente funcionan en la memoria sino que también nos pueden servir para mover información de la memoria a un dispositivo de almacenamiento masivo. Este dispositivo generalmente será el disco duro. Podremos *crear, escribir y leer archivos* en el disco duro apoyándonos en los streams.

Para poder operar con archivos hemos de usar la clase **FileStream**. El *constructor* de esta clase necesita dos parámetros:

- Una cadena que contenga *el nombre del archivo* con el que vamos a trabajar. Es útil que el nombre del archivo también tenga su extensión (*txt, doc,...*).
- El *modo del archivo* que indica cómo funcionará y se manipulará el archivo. El valor colocado debe ser del tipo de enumerado `FileMode` y los valores posibles son:
 - ❖ **Append**: si el archivo existe se abrirá y permitirá añadir nueva información al final del mismo.
 - ❖ **Create**: nos permite crear un nuevo archivo. En el caso de que el archivo ya exista, se sobrescribe.
 - ❖ **CreateNew**: nos permite crear un nuevo archivo. Si el archivo ya existe se produce una excepción.
 - ❖ **Open**: nos permite abrir un archivo que ya existe. Si no existe, se produce una excepción.
 - ❖ **OpenOrCreate**: nos permite abrir un archivo. Si no existe, se crea.
 - ❖ **Truncate**: abre el archivo y elimina su contenido.

Las principales operaciones que podemos realizar con un **FileStream** serán de *escritura y lectura*. A continuación se detallan los métodos que nos permitirán realizar tales operaciones:

- ❖ **Write(array, desplazamiento, numBytes)**. Escribe en el *FileStream* el array de bytes con la información que se va a escribir desde la posición indicada por *desplazamiento* dentro del array. De dicha información se va a coger el número de bytes indicados por *numBytes*.
- ❖ **Read(array, desplazamiento, numBytes)**. Lee del *FileStream* un número de bytes indicado por *numBytes*, los guarda en el array a partir de la posición indicada por *desplazamiento*.

- ❖ **Close()** . Este método ha de utilizarse cuando vayamos a terminar de usar el objeto *FileStream* instanciado.

EJEMPLO:

```
using System;
using System.IO;
using System.Text;

namespace EjArchivos
{
    class Program
    {
        static void Main(string[] args)
        {
            //Abro un archivo llamado miArchivo.txt. Si ya existe, lo sobrescribe
            FileStream fsEscribir = new FileStream("miArchivo.txt", FileMode.Create);

            string cadena = "Esta es la frase 1";

            //Escribo todos los caracteres de la cadena desde la posición 5 del array
            fsEscribir.Write(ASCIIEncoding.ASCII.GetBytes(cadena),5,cadena.Length-5);
            fsEscribir.Close();

            //Cambio el modo de apertura y añado más texto
            fsEscribir = new FileStream("miArchivo.txt", FileMode.Append);

            string cadena2 = "\nEsta es la frase 2";

            fsEscribir.Write(ASCIIEncoding.ASCII.GetBytes(cadena2),0,cadena2.Length);
            fsEscribir.Close();

            //array para guardar el contenido del archivo
            byte[] infoArchivo = new byte[100];

            //Leo el archivo y lo guardo a partir de la posición 2 de infoArchivo
            FileStream fs = new FileStream("miArchivo.txt", FileMode.Open);
            fs.Read(infoArchivo, 2, (int)fs.Length);

            //Muestro el contenido del array
            Console.WriteLine(ASCIIEncoding.ASCII.GetString(infoArchivo));

            //Compruebo lo que se ha guardado en cada posición del array
            for(int i=0; i<(cadena.Length - 5 + cadena2.Length + 2); i++)
            {
                Console.WriteLine($"infoArchivo[{i}]=
                {Convert.ToChar(infoArchivo[i]).ToString()}");
            }
            Console.ReadKey();
            fs.Close();

            //Borro el contenido del archivo
            fs = new FileStream("miArchivo.txt", FileMode.Truncate);
            fs.Close();
        }
    }
}
```